NATIONAL UNIVERSITY OF HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF COMPUTER ENGINEERING

CIRCUIT DESIGN WITH HDL

CHAPTER 4: BEHAVIORAL MODELING

Lecturer: Ho Ngoc Diem

Course outline

- Chapter 1: Introduction
- Chapter 2: Verilog Language Concepts
- Chapter 3: Structural modeling
- Chapter 4: Behavioral modeling
- Chapter 5: State machines
- Chapter 6: Tasks and Functions
- Chapter 7: Functional Simulation/Verification (Testbench)
- Chapter 8: Synthesis of Combinational and Sequential Logic
- Chapter 9: Post-synthesis design tasks
- Chapter 10: VHDL introduction

Behavioral model

- **What is Behavioral model ?**

– Program describes input/output behavior of circuit, tell what you want to have happen, NOT what gates to connect to make it happen.

– Describe what a component does, not how it does

– Many structural models could have the same behavior (different implementations of one expression)

– Synthesized into a circuit that has this behavior

– Result is only as good as the tools

Behavioral model

- **Why behavioral model?**

– Good for more abstract models of circuits

– Easier to write

– More flexible

– Provide sequencing

– A much easier way to write testbenches

– Allow both model and the testbench to be described together

Content

- Boolean-equation-based behavioral model
- Continuous assignment
- Expressions in Verilog
- Cyclic behavioral models with always block
- initial & always construct
- Procedural assignment
- Blocking and non-blocking assignment
- Sequential and Parallel blocks
- Timing control mechanism in Verilog
- Conditional, Case, Loop Statements
- Related modeling techniques

Content

- Boolean-equation-based behavioral model
- Continuous assignment
- Expressions in Verilog
- Cyclic behavioral models with always block
- **initial** & **always** construct
- Procedural assignment
- Blocking and non-blocking assignment
- Sequential and Parallel blocks
- Timing control mechanism in Verilog
- Conditional, Case, Loop Statements
- Related modeling techniques

Boolean-equation-based behavioral model

- For complex design: number of gates is very large -> need a more effective way to describe circuit
- Level of abstraction is higher than gate-level, describe the design using expressions instead of primitive gates

```
module adder (sum, carry_out, carry_in, ina, inb);
output [3:0] sum;
output carry_out;
input [3:0] ina, inb;
input carry_in;
wire carry_out, carry_in;
wire [3:0] sum, ina, inb;

assign {carry_out, sum} = ina + inb + carry_in;
endmodule
```

Continuous assignment

- Drive a value onto a net

  assign out = i1 & i2; //out is net; i1 and i2 are nets

Left-hand side	Right-hand side
Net (vector or scalar) Bit-select or part-select of a vetor net Concatenation of any of the above	Net, register, function call (any expression that gives a value)

- Always active
- Delay value: control time when the net is assigned value

  assign #10 out = in1 & in2; //delay of performing computation,
  //only used by simulator, not synthesis

Continuous assignment

Examples:

wire out = in1 & in2; *//scalar net*

//implicit continuous assignment, declared only once

assign addr[15:0] = addr1_bits[15:0] ^ addr2_bits[15:0]; *//vector net*

assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in; *//concatenation*

```
module adder (sum, carry_out, carry_in, ina, inb);
output [3:0] sum;
output carry_out;
input [3:0] ina, inb;
input carry_in;
wire carry_out, carry_in;
wire [3:0] sum, ina, inb;

assign {carry_out, sum} = ina + inb + carry_in;
endmodule
```

Continuous Assignment

- Because the assignment is done always, exchanging the written order of the lines of continuous assignments has no influence on the logic
- Common error

  - Not assigning a wire a value

  - Assigning a wire a value more than one
- Target (LHS) is NEVER a **reg** variable

Delay

Regular assignment delay

```
assign #10 out = in1 & in2; // Delay in a continuous assign
```

Implicit continuous assignment delay

```
wire #10 out = in1 & in2;

//same as
wire out;
assign #10 out = in1 & in2;
```

Net declaration delay

```
//Net Delays
wire # 10 out;
assign out = in1 & in2;

//The above statement has the same effect as the following.
wire out;
assign #10 out = in1 & in2;
```

Expression: Operands

- Constant number or string
- Parameter
- Net
- Variable (reg, integer, time, real, realtime)
- Array element
- Bit-select or part-select (not for real, realtime)
- Function call that returns any of the above

Constant

Data types

Expression: Operators

Arithmetic	+ - * / % **
Logical	! && ||
Logical equality	== !=
Case equality	=== !===
Bitwise	~ & | ^ ^~ (or ~^)
Relational	< > >= <=
Unary reduction	& ~& | ~| ^ ^~ (or ~^)
Shift	<< >> <<< >>>
Concatenation	{}
Replication	{{}}
Condition	?:
Unary	+ -

Operators not allowed for real expression

Ref. book for detail of each operator!

Arithmetic Operators

- Binary: +, -, *, /, % (the modulus operator)
- Unary: +, - (This is used to specify the sign)
- Integer division truncates any fractional part
- The result of a modulus operation takes the sign of the first operand
- If any operand bit value is the unknown value x, then the entire result value is x
- Register data types are used as unsigned values (Negative numbers are stored in two's complement form)

a + b	a plus b
a - b	a minus b
a * b	a multiplied by b (or a times b)
a / b	a divided by b
a % b	a modulo b
a ** b	a to the power of b

Arithmetic Operators (cont.)

Example

```
 1 module arithmetic_operators();
 2
 3 initial begin
 4    $display ("5 + 10 = %d", 5  + 10);
 5    $display ("5 - 10 = %d", 5  - 10);
 6    $display ("10 - 5 = %d", 10 - 5);
 7    $display ("10 * 5 = %d", 10 * 5);
 8    $display ("10 / 5 = %d", 10 / 5);
 9    $display ("10 / -5 = %d", 10 / -5);
10    $display ("10 %s 3 = %d","%", 10 % 3);
11    $display ("+5     = %d", +5);
12    $display ("-5     = %d", -5);
13    #10  $finish;
14 end
15
16 endmodule
```

Logical Operators

Operator	Description
!	logic negation
&&	logical and
||	logical or

- *Return a **single bit 1 (true), 0 (false), x (unknown, if any operand has bits x)**.*
- *Treat all values that are nonzero as "1"*

Example:

```
1  module logical_operators();
2
3  initial  begin
4     // Logical AND
5     $display  ("1'b1 && 1'b1 = %b",  (1'b1 && 1'b1));
6     $display  ("1'b1 && 1'b0 = %b",  (1'b1 && 1'b0));
7     $display  ("1'b1 && 1'bx = %b",  (1'b1 && 1'bx));
8     // Logical OR
9     $display  ("1'b1 || 1'b0 = %b",  (1'b1 || 1'b0));
10    $display  ("1'b0 || 1'b0 = %b",  (1'b0 || 1'b0));
11    $display  ("1'b0 || 1'bx = %b",  (1'b0 || 1'bx));
12    // Logical Negation
13    $display  ("! 1'b1     = %b",  (!   1'b1));
14    $display  ("! 1'b0     = %b",  (!   1'b0));
15    #10  $finish;
16 end
17
18 endmodule
```

Logical and Case equality Operators

Operator	Description
a === b	a equal to b, including x and z (Case equality)
a !== b	a not equal to b, including x and z (Case inequality)
a == b	a equal to b, result may be unknown (logical equality)
a != b	a not equal to b, result may be unknown (logical equality)

- For the == and != operators:

  - The result is 0, if two bits in two operands that have the same position is different in 0 and 1

    Ex. 4'b1z00 == 4'b1x10 → 0

    4'bxxx1 == 4'bzzz0 → 0

  - The result is x, if either operand contains an x or a z (except the above case)

    Ex. 4'b1z00 == 4'b1000 → x

    4'b1x00 == 4'b1x00 → x

- For the === and !== operators

  bits with x and z are included in the comparison and must match for the result to be true.

Logical and Case equality Operators (cont.)

Operator	Description
a === b	a equal to b, including x and z (Case equality)
a !== b	a not equal to b, including x and z (Case inequality)
a == b	a equal to b, result may be unknown (logical equality)
a != b	a not equal to b, result may be unknown (logical equality)

Example 1:

```
1  module equality_operators();
2
3  initial begin
4     // Case Equality
5     $display ("4'bx001 === 4'bx001 = %b", (4'bx001 ===  4'bx001));
6     $display ("4'bx0x1 === 4'bx001 = %b", (4'bx0x1 ===  4'bx001));
7     $display ("4'bz0x1 === 4'bz0x1 = %b", (4'bz0x1 ===  4'bz0x1));
8     $display ("4'bz0x1 === 4'bz001 = %b", (4'bz0x1 ===  4'bz001));
9     // Case Inequality
10    $display ("4'bx0x1 !== 4'bx001 = %b", (4'bx0x1 !==  4'bx001));
11    $display ("4'bz0x1 !== 4'bz001 = %b", (4'bz0x1 !==  4'bz001));
12    #10 $finish;
13 end
14
15 endmodule
```

Example 2:

4'b1x00 == 4'b1x00
4'b1x00 === 4'b1x00
4'b1100 == 4'b1100
4'b0000 == 4'b0000

4'b1z00 == 4'b1x00
4'b1z00 == 4'b1z00
4'b1100 == 4'b1z10
4'b1100 == 4'b1x10

4'bxxx0 == 4'bzzz1
4'b1z00 == 4'b1x10

Relational Operators

Operator	Description
a < b	a less than b
a > b	a greater than b
a <= b	a less than or equal to b
a >= b	a greater than or equal to b

- Compare two operands *and return a single bit 1 or 0*.
- Be synthesized into comparators.
- The result is x if any of the operands has an unknown (x) or high impedance (z)

Example:

```
1  module relational_operators();
2
3  initial begin
4     $display (" 5    <= 10 = %b", (5      <= 10));
5     $display (" 5    >= 10 = %b", (5      >= 10));
6     $display (" 1'bx <= 10 = %b", (1'bx   <= 10));
7     $display (" 1'bz <= 10 = %b", (1'bz   <= 10));
8     #10  $finish;
9  end
10
11 endmodule
```

Bit-wise Operators

Operator	Description
~	negation
&	and
|	inclusive or
^	exclusive or
^~ or ~^	exclusive nor (equivalence)

Bit-to-bit combination between two operand

- Computations include unknown bits, in the following way:
  - ~x = x
  - 0&x = 0
  - 1&x = x&x = x
  - 1|x = 1
  - 0|x = x|x = x
  - 0^x = 1^x = x^x = x
  - 0^~x = 1^~x = x^~x = x
- When operands are of unequal bit length, the shorter operand is zero-filled in the most significant bit positions.

- ~z = x
- (0,1, x, z) & z = x
- (0,1, x, z) | z = x

Bit-wise Operators (cont.)

Operator	Description
~	negation
&	and
|	inclusive or
^	exclusive or
^~ or ~^	exclusive nor (equivalence)

Example:

```
1  module bitwise_operators();
2
3  initial  begin
4     // Bit Wise Negation
5     $display  (" ~4'b0001       = %b", (~4'b0001));
6     $display  (" ~4'bx001       = %b", (~4'bx001));
7     $display  (" ~4'bz001       = %b", (~4'bz001));
8     // Bit Wise AND
9     $display  (" 4'b0001 & 4'b1001 = %b", (4'b0001 & 4'b1001));
10    $display  (" 4'b1001 & 4'bx001 = %b", (4'b1001 & 4'bx001));
11    $display  (" 4'b1001 & 4'bz001 = %b", (4'b1001 & 4'bz001));
12    // Bit Wise OR
13    $display  (" 4'b0001 | 4'b1001 = %b", (4'b0001 |  4'b1001));
14    $display  (" 4'b0001 | 4'bx001 = %b", (4'b0001 |  4'bx001));
15    $display  (" 4'b0001 | 4'bz001 = %b", (4'b0001 |  4'bz001));
16    // Bit Wise XOR
17    $display  (" 4'b0001 ^ 4'b1001 = %b", (4'b0001 ^ 4'b1001));
18    $display  (" 4'b0001 ^ 4'bx001 = %b", (4'b0001 ^ 4'bx001));
19    $display  (" 4'b0001 ^ 4'bz001 = %b", (4'b0001 ^ 4'bz001));
20    // Bit Wise XNOR
21    $display  (" 4'b0001 ~^ 4'b1001 = %b", (4'b0001 ~^ 4'b1001));
22    $display  (" 4'b0001 ~^ 4'bx001 = %b", (4'b0001 ~^ 4'bx001));
23    $display  (" 4'b0001 ~^ 4'bz001 = %b", (4'b0001 ~^ 4'bz001));
24    #10  $finish;
25 end
26
27 endmodule
```

Unary Reduction Operators

Operator	Description
&	and
~&	nand
|	or
~|	nor
^	xor
^~ or ~^	xnor

- Reduction operators are unary.
- They perform a bit-wise operation on a single operand to produce a single bit result.
- Reduction unary NAND and NOR operators operate as AND and OR respectively, but with their outputs negated.
  - Unknown bits are treated as described before.

Unary Reduction Operators (cont.)

Operator	Description
&	and
~&	nand
|	or
~|	nor
^	xor
^~ or ~^	xnor

Example:

```
1  module reduction_operators();
2
3  initial  begin
4     // Bit Wise AND reduction
5     $display  (" &  4'b1001 = %b", (&   4'b1001));
6     $display  (" &  4'bx111 = %b", (&   4'bx111));
7     $display  (" &  4'bz111 = %b", (&   4'bz111));
8     // Bit Wise NAND reduction
9     $display  (" ~& 4'b1001 = %b", (~& 4'b1001));
10    $display  (" ~& 4'bx001 = %b", (~& 4'bx001));
11    $display  (" ~& 4'bz001 = %b", (~& 4'bz001));
12    // Bit Wise OR reduction
13    $display  (" |  4'b1001 = %b", (|   4'b1001));
14    $display  (" |  4'bx000 = %b", (|   4'bx000));
15    $display  (" |  4'bz000 = %b", (|   4'bz000));
16    // Bit Wise NOR reduction
17    $display  (" ~| 4'b1001 = %b", (~|  4'b1001));
18    $display  (" ~| 4'bx001 = %b", (~|  4'bx001));
19    $display  (" ~| 4'bz001 = %b", (~|  4'bz001));
20    // Bit Wise XOR reduction
21    $display  (" ^  4'b1001 = %b", (^   4'b1001));
22    $display  (" ^  4'bx001 = %b", (^   4'bx001));
23    $display  (" ^  4'bz001 = %b", (^   4'bz001));
24    // Bit Wise XNOR
25    $display  (" ~^ 4'b1001 = %b", (~^  4'b1001));
26    $display  (" ~^ 4'bx001 = %b", (~^  4'bx001));
27    $display  (" ~^ 4'bz001 = %b", (~^  4'bz001));
28    #10  $finish;
29 end
30
31 endmodule
```

Shift Operators

Unsigned data type

Operator	Description
<<	left shift
>>	right shift

- The left operand is shifted by the number of bit positions given by the right operand.
- The vacated bit positions are filled with zeroes.

Signed data type

- Verilog-2001 adds arithmetic shift operators
  - The >>> token does an arithmetic shift right, filling with the value of the sign bit
    - Different than the >> bit shift right operator, which always fills with zero
  - The <<< token does an arithmetic shift left, filling with zeros
    - Same functionality as the << bit shift left operator

Shift Operators (cont.)

Example:

```
1  module signed_shift();
2
3  reg [7:0] unsigned_shift;
4  reg [7:0] signed_shift;
5  reg signed [7:0] data = 8'hAA;
6
7  initial begin
8    unsigned_shift = data >> 4;
9    $display ("unsigned shift = %b", unsigned_shift);
10   signed_shift = data >>> 4;
11   $display ("signed  shift = %b", signed_shift);
12   unsigned_shift = data << 1;
13   $display ("unsigned shift = %b", unsigned_shift);
14   signed_shift = data <<< 1;
15   $display ("signed  shift = %b", signed_shift);
16   $finish;
17 end
18
19 endmodule
```

Concatenation Operators

- Concatenations are expressed using the brace characters { and }, with commas separating the expressions within.
  - Example: + {a, b[3:0], c, 4'b1001} // if a and c are 8-bit numbers, the results has 24 bits
- Unsized constant numbers are not allowed in concatenations.

Example:

```
1  module concatenation_operator();
2
3  initial  begin
4     // concatenation
5     $display  (" {4'b1001,4'b10x1} = %b", {4'b1001,4'b10x1});
6     #10  $finish;
7  end
8
9  endmodule
```

Replication Operators

Operator	Description
{n{m}}	Replicate value m, n times

- Repetition multipliers (must be constants) can be used:
  - {3{a}} // this is equivalent to {a, a, a}
- Nested concatenations and replication operator are possible:
  - {b, {3{c, d}}} // this is equivalent to {b, c, d, c, d, c, d}

Example:

```
1  module replication_operator();
2
3  initial  begin
4     // replication
5     $display  (" {4{4'b1001}}     = %b", {4{4'b1001}});
6     // replication and concatenation
7     $display  (" {4{4'b1001,1'bz}} = %b", {4{4'b1001,1'bz}});
8     #10  $finish;
9  end
10
11 endmodule
```

Conditional Operator

- The conditional operator has the following C-like format:
  - cond_expr ? true_expr : false_expr
- The true_expr or the false_expr is evaluated and used as a result depending on what cond_expr evaluates to (true or false).

Example:

```
// Tri state buffer
assign out = (enable) ? data : 1'bz;
```

Operators

- Examples of basic operators

Example	Results in
`25 * 8'b6`	150
`25 + 8'b7`	32
`25 / 8'b6`	4
`22 % 7`	1
`81b10110011 > 8'b0011`	1
`4'b1011 < 10`	0
`4'b1Z10 < 4'b1100`	X
`4'b1x10 < 4'b1100`	X
`4'b1x10 <= 4'b1x10`	X

Operators

- Examples of Equality operator

Example	Results in
`8'b10110011 == 8'b10110011`	1
`8'b1011 == 8'b00001011`	1
`4'b1100 == 4'b1Z10`	0
`4'b1100 != 8'b100X`	1
`8'b1011 !== 8'b00001011`	0
`8'b101X === 8'b101X`	1

- Examples of Shift operator

Example	Results in
`8'b0110_0111<<3`	`8'b0011-1000`
`8'b0110_0111<<1'bz`	`8'bxxxx-xxxx`
`Signed_LHS = 8'b1100-0000>>>2`	`8'b1111-0000`

Operators

- Logical, Bit-wise, Reduction operator

Example	Results in
`8'b01101110 && 4'b0`	0
`8'b01101110 || 4'b0`	1
`8'b01101110 && 8'b10010001`	1
`! (8'b10010001)`	1
`8'b01101110 & 8'bxxzz1100`	`8'b0xx01100`
`8'b01101110 | 8'bxxzz1100`	`8'x11x1110`
`~& (4'b0xz1)`	1
`~| (4'b0xz1)`	0

Operator precedence

+ - ! ~ & ~& | ~| ^ ~^ ^~ (unary)	Highest precedence
**	
* / %	
+ - (binary)	
<< >> <<< >>>	
< <= > >=	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
| (binary)	
&&	
||	
?: (conditional operator)	
{} {{}}	Lowest precedence

Expression example

- Bitwise operator

```
module xor3 (input a, b, c, output y);
assign y = a ^ b ^ c;
endmodule
```

- Concatenation

```
module add_1bit (input a, b, ci, output s, co);
assign #(3, 4) {co, s} = {(a & b)|(b & ci)|(a & ci), a^b^ci};
endmodule
```

- Conditional operator

```
module quad_mux2_1 (input [3:0] i0, i1, input s, output [3:0] y);
assign y = s ? i1 : i0;
endmodule
```

Expression example

- Relational & Equality operator

```
module comp_4bit ( input [3:0] a, b, output a_gt_b, a_eq_b, a_lt_b);
assign a_gt_b = (a>b),
       a_eq_b = (a==b),
       a_lt_b = (a<b);
endmodule
```

- Arithmetic operator

```
module add_4bit (input [3:0] a, b, input ci, output [3:0] s, output co);
assign { co, s } = a + b + ci;
endmodule
```

Combinational circuit

- 4 to 1 mux

```
module mux4_1(out, in1, in2, in3 ,in4,
                          cntrl1, cntrl2);
output out;
input in1, in2, in3, in4, cntrl1, cntrl2;
assign out =  (in1 & ~cntrl1 & ~cntrl2) |
              (in2 & ~cntrl1 &  cntrl2)  |
              (in3 &  cntrl1 & ~cntrl2)  |
              (in4 &  cntrl1 &  cntrl2);
endmodule
```

```
module mux4_1 (out, in1, in2, in3, in4,
                                  cntrl) ;
output out ;
input in0,in1,in2,in3 ;
input [1:0] cntrl;
assign out = (cntrl == 2'b00) ? in0 :
             (cntrl == 2'b01) ? in1 :
             (cntrl == 2'b10) ? in2 :
             (cntrl == 2'b11) ? in3 :
              1'bx ;
endmodule
```

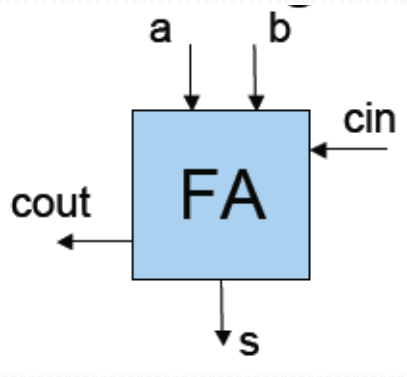
OR

```
// Use nested conditional operator
assign out = cntrl1 ? (cntrl2 ? in4 : in3) : (cntrl2 ? in2 : in1);
```

Combinational circuit

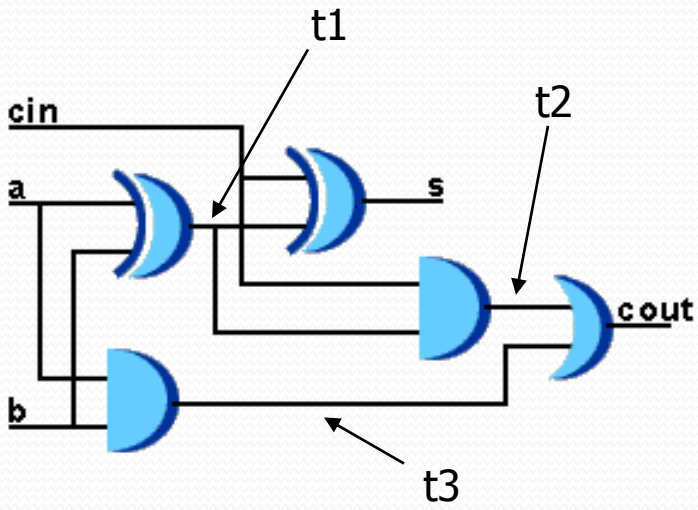
- 1 bit full adder

```
module fa (input a, b, cin,
        output s, cout);
assign s = a^b^cin;
assign cout = (a & b) | (cin & (a^b));
endmodule
```



- Let's design 8-bit adder

```
module adder(cout,s,a,b) ;
output cout;
output [7:0] s ;
input [7:0] a,b ;
assign {cout,s} = a + b ;
endmodule
```



Combinational circuit

- Comparator

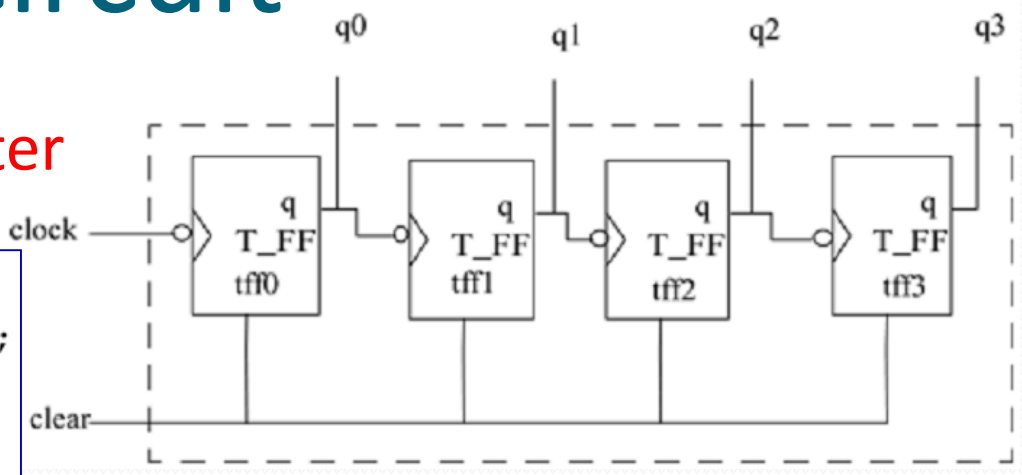
Comparator makes the comparison A ? B
"?" is determined by the input greaterNotLess
and returns true(1) or false(0).

Parameters that may be set
when the module is instantiated.

```
module comparator (result, A, B, greaterNotLess);
   parameter width = 8;
   parameter delay = 1;
   input [width-1:0] A, B;          // comparands
   input greaterNotLess;             // 1 - greater, 0 - less than
   output result;                   // 1 if true, 0 if false

   assign #delay result = greaterNotLess ? (A > B) : (A < B);
endmodule
```

Sequential circuit

- 4-bit ripple carry counter

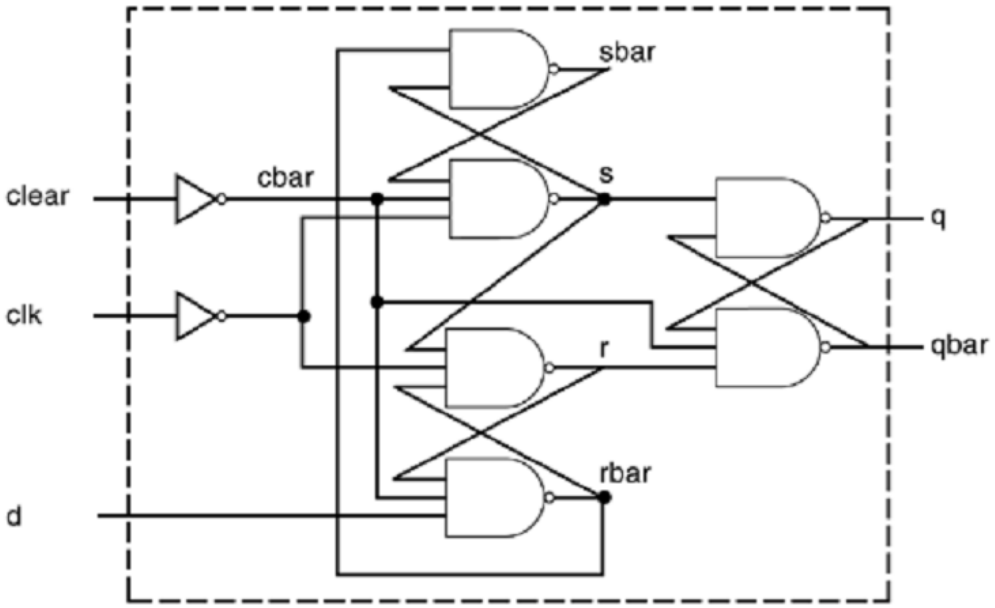



```
// Ripple counter
module counter(Q , clock, clear);

// I/O ports
output [3:0] Q;
input clock, clear;

// Instantiate the T flipflops
T_FF tff0(Q[0], clock, clear);
T_FF tff1(Q[1], Q[0], clear);
T_FF tff2(Q[2], Q[1], clear);
T_FF tff3(Q[3], Q[2], clear);

endmodule
```

```
module T_FF(q, clk, clear);

// I/O ports
output q;
input clk, clear;

// Instantiate the edge-triggered DFF
// Complement of output q is fed back.
// Notice qbar not needed. Unconnected port
edge_dff ff1(q, ,~q, clk, clear);

endmodule
```

Sequential circuit

- 4-bit ripple carry counter




```
// Edge-triggered D flipflop
module edge_dff(q, qbar, d, clk, c

// Inputs and outputs
output q,qbar;
input d, clk, clear;

// Internal variables
wire s, sbar, r, rbar,cbar;
assign cbar = ~clear;

// Input latches; A latch is level sensitive. An edge-sensitive
// flip-flop is implemented by using 3 SR latches.
assign  sbar = ~(rbar & s),
        s = ~(sbar & cbar & ~clk),
        r = ~(rbar & ~clk & s),
        rbar = ~(r & cbar & d);

// Output latch
assign  q = ~(s & qbar),
        qbar = ~(q & r & cbar);
endmodule
```

Summary

- Continuous assignment: main construct in dataflow modeling, always active
- Left hand side of assignment must be a net
- Using expression with operators & operands
- Delays on a net can be defined in the assign statement, implicit continuous assignment, or net declaration.
- To describe much sophisticated logic easily, procedural assignment is available in Verilog RTL programming (see later)